

```

In [1]: import numpy as np
import modern_robotics as mr
import csv

def IKinBodyIterates(Blist, M, T, thetalist0, eomg, ev):
    """
    ニュートン・ラプソン法

    IN:
    Blist      : ボディスクリュー軸の行列
    M          : 初期状態でのロボットアーム先端の位置姿勢
    T          : 目標のロボットアーム先端の位置姿勢
    thetalist0 : 初期の関節角
    eomg       : 回転エラーの閾値
    ev         : 並進エラーの閾値

    OUT:
    thetalist  : 最終的な関節角
    success    : 収束した場合はTrue
    """
    # 初期値
    thetalist = np.array(thetalist0, dtype=float)
    maxiterations = 10
    i = 0

    # 関節角の履歴
    joint_hist = []

    # ロボットアーム先端の座標系の位置と姿勢
    Tsb = mr.FKinBody(M, Blist, thetalist)
    # 目標座標系tから見たロボットアーム先端のずれ
    Tbt = np.dot(mr.TransInv(Tsb), T)
    # ツイストへの変換
    Vb = mr.se3ToVec(mr.MatrixLog6(Tbt))

    # 終了条件の誤差チェック (回転エラー/並進エラー)
    err = np.linalg.norm(Vb[0:3]) > eomg or np.linalg.norm(Vb[3:6]) > ev

    while err and i < maxiterations:
        # 現在の関節角を履歴に追加
        joint_hist.append(thetalist.copy())

        # 進行状況の表示
        print(f"%nIteration {i}:")
        print("%njoint vector:")
        print(thetalist)
        print("%nSE(3) end-effector configuration:")
        print(Tsb)
        print("%nerror twist V_b:")
        print(Vb)
        print("%nangular error magnitude:")
        print(np.linalg.norm(Vb[0:3]))
        print("%nlinear error magnitude:")
        print(np.linalg.norm(Vb[3:6]))

        # ヤコビ行列の計算
        Jb = mr.JacobianBody(Blist, thetalist)
        # Vbだけ動かす場合の関節角の変位角
        thetalist += np.linalg.pinv(Jb) @ Vb

        i += 1

```

```

# 再計算
Tsb = mr.FKinBody(M, Blist, thetalist)
Vb = mr.se3ToVec(mr.MatrixLog6(np.dot(mr.TransInv(Tsb), T)))
err = np.linalg.norm(Vb[0:3]) > eomg or np.linalg.norm(Vb[3:6]) > ev

# 関節角の履歴 (最終値の追加)
joint_hist.append(thetalist.copy())

# CSVファイルへの保存
with open('iterates.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerows(joint_hist)

success = not err

return thetalist, success

```

```

In [2]: import numpy as np
import modern_robotics as mr

# 定義値
w1=0.109
w2=0.082
l1=0.425
l2=0.392
h1=0.089
h2=0.095

Blist = np.array([[0, 0, 0, 0, 0, 0],
                  [1, 0, 0, 0, -1, 0],
                  [0, 1, 1, 1, 0, 1],
                  [w1+w2, h2, h2, h2, -w2, 0],
                  [0, -l1-l2, -l2, 0, 0, 0],
                  [l1+l2, 0, 0, 0, 0, 0]])

M = np.array([[-1, 0, 0, l1+l2],
              [0, 0, 1, w1+w2],
              [0, 1, 0, h1-h2],
              [0, 0, 0, 1]])

# 目標値
T = np.array([[0, 1, 0, -0.5],
              [0, 0, -1, 0.1],
              [-1, 0, 0, 0.1],
              [0, 0, 0, 1]])

# 初期値
thetalist0 = np.array([2, -0.5, 1.3, 1.7, 0.3, 1.2])

# 誤差の閾値
eomg = 0.001 # 向き
ev = 0.0001 # 位置

```

```

In [ ]: IKinBodyIterates(Blist, M, T, thetalist0, eomg, ev)

```